
What the Final Patch Hides: Event-Level Regression Measurement for Coding-Agent Harnesses

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Coding agents are evaluated on the patch they leave behind. We show that this
2 final state hides most of what happens during a run. We built an instrument that
3 records every mutating action of an agent as a git commit on a hidden reference,
4 replays each benchmark instance's held-out tests at every recorded state inside
5 the benchmark's own container image, and attributes each detected regression to
6 the harness check that could have caught it. Applied to SWE-bench Verified, the
7 instrument recorded 184 regression events across the runs that produced any; the
8 final patch exposed one of them. Regression frequency depended on the model
9 stack rather than the benchmark: on the same 40 instances under the same harness,
10 one frontier stack produced no events in 40 runs and another produced 140 events in
11 11 incidents (Fisher $p = 0.010$). A pre-registered comparison of recovery policies,
12 analysed with code committed before unblinding, found that rollback to a verified
13 checkpoint left contamination in one final tree where no recovery left it in nine
14 (paired sign test, $p = 0.022$), but rollback also resolved fewer tasks, because the
15 verifier that triggers it rejected patches the official grader accepted. Replacing that
16 verifier with the repository's own tests removed the tradeoff: 65% resolved with no
17 contaminated final tree (McNemar $p = 0.012$ against plain rollback). We release
18 the instrument, its validation protocol, and a catalogue of 28 measurement defects
19 encountered while building it, 23 of which produced plausible numbers rather than
20 errors.

21 1 Introduction

22 Benchmarks for coding agents grade the final patch. SWE-bench [1] and its Verified subset [2] run
23 the repository's tests against the patch an agent submits and report whether the target tests now pass
24 and the previously passing tests still pass. Work on regressions introduced by agents follows the same
25 convention: it examines the submitted patch and counts the previously passing tests it breaks [8].

26 This convention has a blind spot. A regression that an agent introduces and later repairs leaves no
27 trace in the final patch, and any harness with a recovery mechanism produces that pattern by design.
28 Whether it matters depends on two unanswered empirical questions: how often regressions occur
29 during a run, and how much of that activity survives to the final state.

30 We answer both by measuring the timeline rather than the endpoint. Our contributions are:

- 31 **1. An instrument for event-level regression measurement.** Every mutating tool call is
32 recorded as a commit on a git reference the agent cannot see; after the run, the instance's
33 held-out passing tests are replayed at every recorded state inside the benchmark's pinned

34 container, and detection is attributed by coverage. Validity is established by negative and
35 positive controls, a liveness gate, a golden check that drives the gold patch through the
36 agent’s execution path, and a re-scoring control that re-measures archived runs with no
37 model calls (Section 3).

38 2. **Measurements on SWE-bench Verified.** Across the GPT-5.6 rollback runs that produced
39 any regression, the timeline recorded 184 events and the final state exposed one (Section
40 5.2). Event frequency depended on the model stack: identical instances and harness gave 0
41 events under one frontier stack and 140 under another (Section 5.3).

42 3. **A pre-registered comparison of recovery policies, and a fix.** With the analysis committed
43 before unblinding, rollback to a verified checkpoint left fewer contaminated final trees than
44 no recovery ($p = 0.022$) but resolved fewer tasks; we trace the loss to verifier precision,
45 and a post hoc arm with a regression-gated verifier built from the instrument’s own oracle
46 removes it, reaching 65% resolve with no contaminated tree (Sections 5.4 to 5.6).

47 4. **A catalogue of 28 measurement defects** found while building the instrument, classified by
48 mechanism, with the design rules that caught most of them (Appendix A); 23 produced a
49 plausible number rather than an error.

50 All measurements are exploratory and were made on a development slice of the benchmark that is
51 excluded from any confirmatory frame. No claim is made about the population of instances beyond
52 that slice.

53 2 Background and related work

54 **Benchmarks and their validity.** SWE-bench [1] grades a patch by tests that must go from failing
55 to passing and tests that must keep passing; Verified [2] is the human-screened 500-instance subset.
56 Concerns about the static subset have accumulated: contamination, memorisation, and leaked
57 solutions [6], flawed tests [7], and rolling [3, 5] or private [4] alternatives in response. UTBoost
58 [7] showed the held-out tests are often insufficient, changing 24% of Verified leaderboard verdicts.
59 TDAD [8] measured regressions in submitted patches on Verified and found them in about 30% of
60 resolved instances; we measure the same quantity on the timeline, and Section 5.2 compares the two
61 on the same runs.

62 **Agent scaffolds and recovery.** SWE-agent [9], OpenHands [10], Agentless [11], and mini-swe-agent
63 [12] span the scaffold design space, on the action formalisms of CodeAct [13] and ReAct [14].
64 Within-episode recovery has been studied as self-reflection [15, 16], progress-gated recovery [17],
65 checkpoint repair [18], and provenance-based rollback [19]; operating-system framings [20] treat
66 checkpointing as a shared primitive. Process reward models and rubric verifiers [24, 25] approach the
67 verifier-precision problem of Section 5.5 from the training side.

68 **Automated program repair.** The regression problem is the plausible-versus-correct patch problem of
69 program repair [26, 28, 29]; regression tests are the main defence against overfitting patches [27]. Our
70 results concern regressions the agent’s own process repairs before the patch is tested; trajectory-level
71 evaluation [21–23] argues more generally that resolve rate alone is uninformative.

72 3 Instrument

73 Figure 1 shows the harness and the instrument. **Observational timeline.** The harness wraps every
74 mutating tool call. After each call, the working tree is committed to a git reference outside the agent’s
75 view, using a private index so that the agent’s own `git status` and `git diff` are unchanged.
76 Rollbacks and the end of the run are recorded as observations too. Because observations are commits,
77 an archived run can be re-measured later without re-running the agent.

78 **Exhaustive replay.** For each observation, the instance’s passing-test set is run inside the pinned
79 container against that observation’s tree. A regression event is a test that passes at one observation
80 and fails at a later one. Every observation is replayed rather than bisected, because bisection assumes
81 monotone verdicts and a recovery policy violates that by construction. Infrastructure failures during
82 replay are recorded as missing observations rather than as test failures, and baseline-dead tests are
83 excluded from event counting.

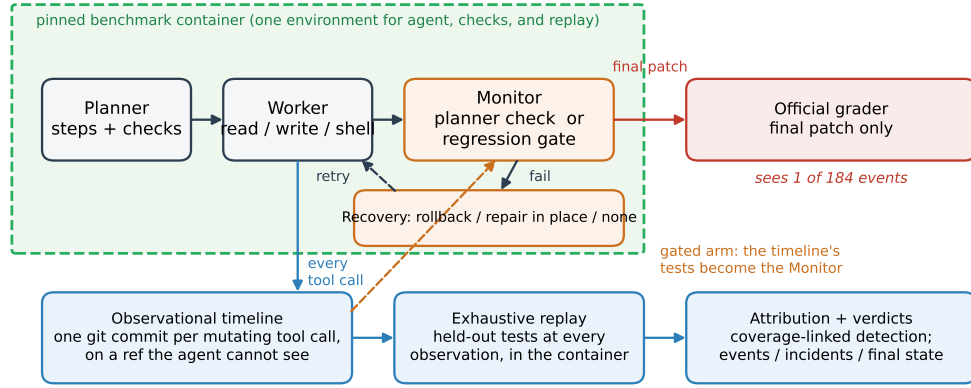


Figure 1: The harness and the instrument. The planner, worker, monitor, and recovery policy run inside the benchmark’s pinned container; every mutating tool call is committed to a hidden git reference; held-out tests are replayed at every observation and detections are attributed by coverage. In the gated arm (dashed), the timeline’s tests serve as the monitor. The official grader sees only the final patch.

84 **Attribution.** A regression is attributed to a harness check if a failing check and the broken held-out
85 test both exercise a file the agent changed at that observation, using a per-instance coverage map built
86 at the base commit; attributed, co-occurring, and unknown are reported separately.

87 **Execution environment.** The agent’s tools and the harness’s checks execute inside the same pinned
88 container as the replay, with file changes synchronised to the host tree the timeline records (Appendix
89 A.2 describes what it cost to learn this).

90 **Validation.** Five gates run before any paid experiment: a negative control, a positive control
91 with injected regressions including recovered ones, a flake screen, an unknown-rate ceiling, and
92 a baseline liveness check that the probes return passes. A golden check drives the benchmark’s
93 gold patch through the agent’s real tool path and requires the grader to return "resolved" (and a null
94 run "unresolved"); it passed on the three repository families with the most distinctive runners. A
95 re-scoring control re-measures an archived run’s committed timeline under a changed instrument
96 with no model calls; on both runs to date it reproduced the archived episode counts.

97 4 Experimental setup

98 **Benchmark and slice.** SWE-bench Verified, 500 instances. A 40-instance development slice (16
99 of them from django), drawn by stratified sampling and fixed before any measurement, was used
100 for all experiments here; it is excluded from the confirmatory frame a later study would use. The
101 GPT-5.6 stack was additionally run on the slice’s first 10 instances as a calibration before the full
102 sweep; where results pool the two, the denominator is stated.

103 **Harness.** A planner decomposes the task into steps with verification commands; a worker executes
104 each step with three tools (read file, write file, run shell); a monitor runs the verification, and on
105 failure the recovery policy acts: **rollback** (reset to the last verified checkpoint, retry with feedback),
106 **repair-in-place** (retry from the failed tree), or **no recovery** (keep the failed step’s tree). Each run has
107 a \$4 work-cost cap; exhaustion is scored as failure.

108 **Models.** Two frontier stacks under identical harness, policy, instances, and caps: Claude (Opus 4.7
109 planner, Sonnet 4.6 worker) and GPT-5.6 (sol planner, terra worker), with identifiers recorded in
110 every run manifest.

111 **Outcomes.** Resolve is the official grader’s verdict on the final patch. Event counts are reported
112 at three levels because the distribution is heavily overdispersed: distinct incidents (observations at

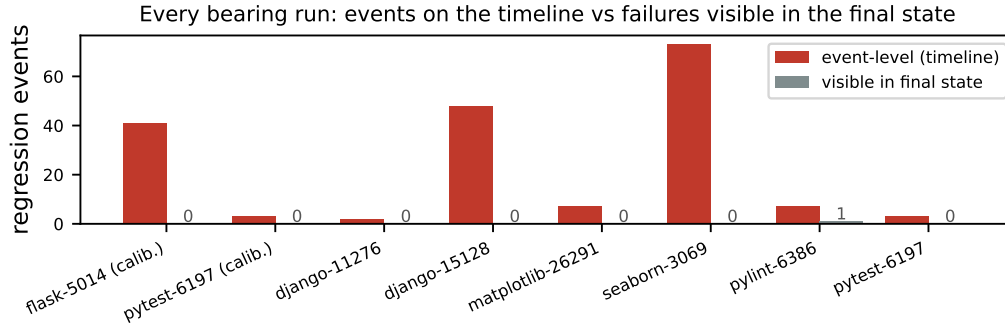


Figure 2: Every run that produced at least one regression event, in both GPT-5.6 sweeps. Left bars (red) are events recorded on the timeline; right bars (grey) are held-out test failures visible in the graded final patch.

which at least one test broke), declared events (one per test function and onset, parametrised variants collapsed), and bearing runs (runs with at least one event). Final-state contamination is the number of held-out tests failing in the graded final patch, net of baseline-dead tests.

Pre-declaration. The event unit, the gates, and the contrast were declared before the relevant data existed; the contrast’s analysis (exact paired sign tests on final-state contamination as primary and incident exposure as co-primary) was committed before either comparison arm finished. One amendment, making the gates conditional on the model stack, preceded unblinding. Appendix B gives the wording.

5 Results

5.1 Resolve and cost

Under rollback, the Claude stack resolved 13 of 40 instances (32.5%, exact 95% CI 18.6% to 49.1%) at \$34.24; the GPT-5.6 stack resolved 13 of 40 (32.5%; 13 of the 35 graded, 37.1%) at \$8.14. Three GPT-5.6 cells were never attempted (the sweep’s circuit breaker stopped after six consecutive zero-progress failures in one repository family) and two were lost to a file-synchronisation defect; all five count as unresolved out of 40 and are excluded from the graded denominator of 35. Paired comparisons below use only instances graded in both arms.

5.2 The final state hides almost all regression activity

Figure 2 shows every run that produced a regression event under the GPT-5.6 stack with the rollback policy, pooling the 10-instance calibration and the 40-instance sweep (47 runs). The timeline recorded 184 declared events across eight runs; the final patch exposed one. Run-weighted, seven of the eight bearing runs ended with a clean final patch. Three storms (73, 48, and 41 events) supply 88% of the event count, so the run-weighted figure is the more robust statement. The same measurement made from the final patch, which is how prior work measures regressions [8], reports one event in eight runs.

Figure 3 shows a single run in detail. The grader marked it resolved with all graded tests passing; its timeline contains three regressions of the same test function at observations 3, 5, and 7, each repaired by rollback. Two of the three were not detected by any harness check while open; the rollback that repaired them was triggered by an unrelated failure. Across the full GPT-5.6 sweep, 48 of 162 raw episodes (events before collapsing parametrised variants) had no co-occurring harness failure at all.

5.3 Regression frequency depends on the model stack, not the benchmark

On the same 40 instances, under the same harness and rollback policy, the Claude stack produced no regression events in 40 runs (227 observations, all replayed). The GPT-5.6 stack produced 140

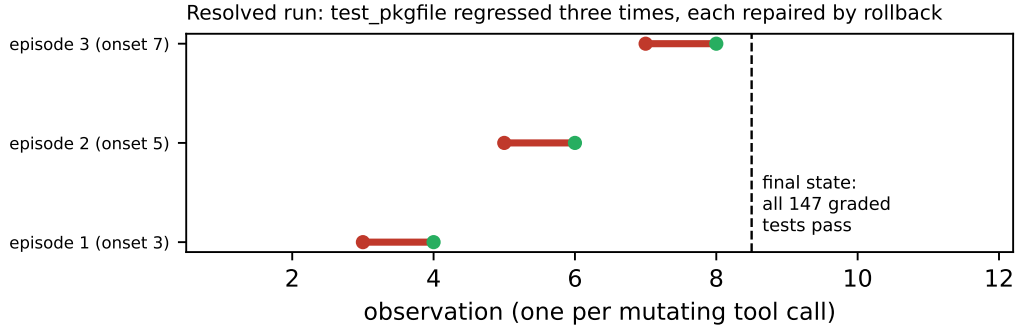


Figure 3: One run that the official grader marked resolved, with all 147 graded tests passing. The timeline shows three regressions of the same test function, each repaired by rollback two observations later.

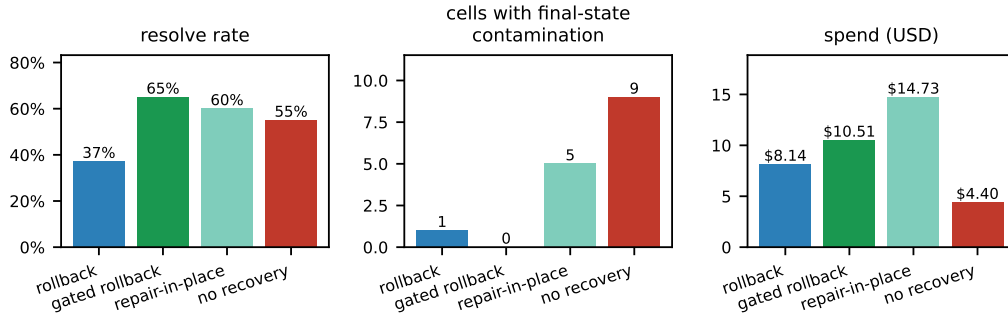


Figure 4: The four recovery arms under the GPT-5.6 stack on the same 40 instances. Contamination counts cells whose final patch fails a previously-passing test.

145 declared events in 11 incidents across 6 of 37 attempted runs (bearing fraction 16.2%, exact CI 6.2%
 146 to 32.0%). A one-sided Fisher exact test on the bearing fractions gives $p = 0.010$. Observation density
 147 differs by a factor of 1.2 between the stacks (5.7 and 7.0 per run), which does not account for a
 148 difference of 140 to 0.

149 The Claude result is a measured zero rather than a detection failure: the same stack’s earlier canary
 150 run produced three events, and re-scoring its archived timeline under the instrument used for the
 151 40-run sweep reproduces them.

152 We had first read the low event rate as a property of the benchmark, and a pre-declared rule directed a
 153 switch to one with a larger held-out set; the second stack falsified that on the same instances. The
 154 pre-declared gate (event rate at least 0.30 per run and bearing fraction at least 25%) nevertheless
 155 failed for both stacks: the GPT-5.6 bearing fraction was 20% at calibration and 16.2% at full scale.
 156 No arm in this paper is a confirmatory pass, and the amendment that made the gates conditional on
 157 the model stack was a response to that failure. In our single-sweep comparison, regression frequency
 158 depended on the model stack (0 of 40 versus 6 of 37); we do not claim more than that.

159 5.4 Recovery policy: rollback keeps the final tree clean

160 Figure 4 summarises the pre-declared contrast. The primary endpoint, final-state contamination, is
 161 paired by instance: no recovery was worse than rollback on 9 instances, better on 1, and tied on
 162 25 (exact sign test, $p = 0.022$). Repair-in-place was worse on 5, better on 1, and tied on 29 ($p =$
 163 0.22). The co-primary endpoint, incident exposure, did not differ detectably for either comparison
 164 (no recovery $p = 0.45$; repair-in-place $p = 1.0$): regressions occurred under every policy at similar
 165 rates, and the policy determined whether they persisted.

166 One no-recovery run illustrates the mechanism: its final tree begins with the literal string \$(cat
167 django/db/models/expressions.py), a shell idiom pasted into a file write, and none of the
168 instance’s 137 graded tests ran; under rollback the same model’s failed attempt was reset and the final
169 patch passed all 137.

170 **Disclosure.** The first unblinding gave $p = 0.375$: seven cells (five no-recovery, two repair-in-place)
171 had been dropped as ungradable because their final trees made the suite uncollectable and the grader
172 returned no result. The official rule scores such a patch as failing every test, so the most contaminated
173 final states had been removed from the endpoint they bore on most. The grader now distinguishes
174 an environment failure from a patch that kills the suite, and the seven archived trees were re-graded
175 without re-running any agent.

176 5.5 Rollback loses resolution to verifier precision

177 Rollback resolved fewer tasks than either alternative (Figure 4, left): 13 of 40 against 24 of 40 for
178 repair-in-place and 22 of 40 for no recovery. Paired by instance, rollback won one discordant pair
179 and lost nine against repair-in-place (McNemar exact $p = 0.022$), and won three and lost nine against
180 no recovery ($p = 0.15$). The loss is attributable to the verifier. Of the 18 rollback runs that failed, 15
181 failed at the first step, and 9 of the 18 were resolved under a policy that kept the rejected work: the
182 monitor’s planner-written check had rejected a patch the grader accepted, and rollback discarded
183 it. Both effects follow from the same mechanism. Rollback discards work whenever the verifier
184 rejects a step, so the verifier’s false-rejection rate determines how many correct patches are lost per
185 contaminated tree avoided.

186 Section 5.6 evaluates replacing the planner-written check with the repository’s own tests.

187 5.6 Regression-gated rollback removes the tradeoff

188 A fourth arm, added after the contrast was unblinded and therefore exploratory, replaces the monitor’s
189 planner-written check with the repository’s previously-passing tests, run in the agent’s container at
190 the start of the run and after every attempt; a step is rejected only if a test that passed at the start fails
191 now. Nothing else changes. It resolved **26 of 40 instances (65.0%)**, the highest of any arm, and left
192 **no contaminated final tree** (Figure 4). Paired against plain rollback on the 35 shared instances, it
193 resolved 10 that plain rollback did not and lost 1 (McNemar exact $p = 0.012$). Paired onset exposure
194 did not differ detectably ($p = 0.73$); the gated arm recorded 70 declared events against 140, a count
195 dominated by a few storms that we do not treat as a finding. The gate’s oracle and the contamination
196 endpoint are the same test set, so the clean trees are partly guaranteed by construction; the informative
197 result is the resolve gain. The three gated failures were genuine regressions the gate refused (8, 2,
198 and every test). Arm costs were \$8.14 (rollback), \$10.51 (gated), \$14.73 (repair-in-place), and \$4.40
199 (no recovery).

200 6 Discussion

201 **Implications for agent-OS design.** Systems that provide checkpointing and recovery as shared
202 primitives should also provide timeline observability. Without it, recovery reduces the number of
203 defects visible in the final state without reducing the number that occur, which removes the signal
204 needed to compare recovery policies.

205 **On the measurement defects.** Building the instrument produced 28 defects (Appendix A); 23
206 produced a plausible number rather than an error, and 27 were present while a passing test suite
207 watched. Two rules would have prevented most of them: record infrastructure failures as missing
208 observations rather than measured outcomes, and require a positive liveness signal, since an empty
209 result is otherwise indistinguishable from a broken measurement.

210 7 Limitations

211 All measurements come from a 40-instance development slice, one sweep per arm plus a 10-instance
212 calibration for the GPT-5.6 stack; run-to-run variability is unmeasured, and the confirmatory design
213 specifies seeded replications on held-out instances. The cross-stack comparison is a single pairwise

test on six bearing runs, confounded with the provider adapter. The gated arm is post hoc. The held-out tests observe roughly the gold test patch’s blast radius, so event counts are lower bounds. Confidence intervals ignore repository clustering (16 of 40 instances are django). The instrument has not yet been applied to a public scaffold.

8 Conclusion

Final-state evaluation of coding agents misses the regressions that the agent’s own process repairs, and in our measurements that is nearly all of them. Measuring the timeline is feasible, validates against controls, and separates recovery policies the final patch cannot distinguish. It also identifies rollback’s cost as a verifier-precision problem, and a verifier built from the same tests resolved more tasks than any other policy while keeping every final tree clean.

References

- [1] C. Jimenez et al. SWE-bench: Can language models resolve real-world GitHub issues? ICLR 2024. arXiv:2310.06770.
- [2] OpenAI. Introducing SWE-bench Verified. 2024.
- [3] L. Zhang et al. SWE-bench Goes Live! arXiv:2505.23419, 2025.
- [4] X. Deng et al. SWE-Bench Pro: Can AI agents solve long-horizon software engineering tasks? arXiv:2509.16941, 2025.
- [5] I. Badertdinov et al. SWE-rebench: An automated pipeline for task collection and decontaminated evaluation of software engineering agents. arXiv:2505.20411, 2025.
- [6] S. Liang et al. The SWE-Bench Illusion: When state-of-the-art LLMs remember instead of reason. arXiv:2506.12286, 2025.
- [7] H. Yu et al. UTBoost: Rigorous evaluation of coding agents on SWE-Bench. ACL 2025. arXiv:2506.09289.
- [8] TDAD: Regressions introduced by coding agents on SWE-bench Verified. arXiv:2603.17973, 2026.
- [9] J. Yang et al. SWE-agent: Agent-computer interfaces enable automated software engineering. NeurIPS 2024. arXiv:2405.15793.
- [10] X. Wang et al. OpenHands: An open platform for AI software developers as generalist agents. ICLR 2025. arXiv:2407.16741.
- [11] C. Xia et al. Agentless: Demystifying LLM-based software engineering agents. arXiv:2407.01489, 2024.
- [12] SWE-agent team. mini-swe-agent. Software, 2025.
- [13] X. Wang et al. Executable code actions elicit better LLM agents. ICML 2024. arXiv:2402.01030.
- [14] S. Yao et al. ReAct: Synergizing reasoning and acting in language models. ICLR 2023. arXiv:2210.03629.
- [15] N. Shinn et al. Reflexion: Language agents with verbal reinforcement learning. NeurIPS 2023. arXiv:2303.11366.
- [16] A. Madaan et al. Self-Refine: Iterative refinement with self-feedback. NeurIPS 2023. arXiv:2303.17651.
- [17] ReflexGrad: Within-episode failure recovery in LLM agents via progress-gated dual-process routing. arXiv:2511.14584, 2025.
- [18] REPOT: Recoverable program-of-thought via checkpoint repair. arXiv:2605.30052, 2026.

- 256 [19] From agent traces to trust: Evidence tracing and execution provenance in LLM agents.
257 arXiv:2606.04990, 2026.
- 258 [20] K. Mei et al. AIOS: LLM agent operating system. arXiv:2403.16971, 2024.
- 259 [21] Holistic Agent Leaderboard: The missing infrastructure for AI agent evaluation.
260 arXiv:2510.11977, 2025.
- 261 [22] What resolve rate hides: Trajectory structure diagnostics for coding agents. arXiv:2607.06184,
262 2026.
- 263 [23] SWE-EVO: Benchmarking coding agents in long-horizon software evolution scenarios.
264 arXiv:2512.18470, 2025.
- 265 [24] Agentic rubrics as contextual verifiers for SWE agents. arXiv:2601.04171, 2026.
- 266 [25] SWE-Shepherd: Advancing PRMs for reinforcing code agents. arXiv:2604.10493, 2026.
- 267 [26] Patch overfitting in program repair: A survey. 2024.
- 268 [27] Y. Wang et al. When automated program repair meets regression testing: An extensive study on
269 two million patches. ACM TOSEM, 2024. arXiv:2105.07311.
- 270 [28] Patch correctness assessment: A survey. ACM TOSEM, 2024. doi:10.1145/3702972.
- 271 [29] H. Ye et al. Automated patch assessment for program repair at scale. Empirical Software
272 Engineering, 2021.
- 273 [30] SWE-bench issue #601: Test result hijacking via stdout forging in the evaluation harness.
274 github.com/SWE-bench/SWE-bench, 2025.
- 275 [31] OpenHands issues #4235 and #7044: Environment errors in SWE-bench instance evaluation.
276 github.com/OpenHands/OpenHands.
- 277 [32] J. Yang et al. SWE-smith: Scaling data for software engineering agents. arXiv:2504.21798,
278 2025.
- 279 [33] J. Pan et al. Training software engineering agents and verifiers with SWE-Gym.
280 arXiv:2412.21139, 2024.
- 281 [34] Context as a tool: Context management for long-horizon SWE agents. arXiv:2512.22087, 2025.

282 **A The measurement defect catalogue**

283 **A.1 The catalogue by mechanism**

284 Each row is a defect found while building the instrument. **S**: it produced a plausible number rather
285 than an error. **G**: it was present while the test suite passed (A4 is the suite). **H**: it was findable only
286 on a clean host. Class A rows depend on the machine the code runs on; class B rows render an
287 infrastructure failure as a measurement; class C rows measure a state where an event was defined;
288 class D rows consume a producer’s output without checking the producer.

#	Defect	Consequence	S	G	H
A1	Shadow commits inherited the machine's git identity	timeline silently empty on any clean machine	✓	✓	✓
A2	Gate probe invoked bare <code>python</code>	every probe exit 127 on a clean host	✗	✓	✓
A3	Unpinned SDK resolved a different major version	two machines ran different code	✗	✓	✓
A4	Test fixtures verified with bare <code>pytest</code> from PATH	15 tests fail on a clean host as harness defects	✗	—	✓
A5	A benchmark scorer invoked bare <code>python</code>	score 0.0 from a missing interpreter	✓	✓	✓
A6	Agent executed on the host; measurement in the container	26 of 28 zero-step runs; see A.2	✓	✓	✓
B1	Output marker used shell :	a 13/13 run scored as 13 errors	✓	✓	
B2	Results on stderr, markers on stdout	one framework's results outside the parsed slice	✓	✓	
B3	Probe diffed against a deleted commit	every graded test a hole	✓	✓	
B4	Interrupted mirror clone accepted with zero commits	later instances fail far from the cause	✓	✓	
B5	Container workdir unvalidated	every exec exit 127	✓	✓	
B6	Isolation removed loopback, not only the internet	23.5% of the oracle dead	✓	✓	
289 B7	Provider cache served removed containers	later cells replay against nothing and score clean	✓	✓	
B8	Tree reset reverted image build-time edits	one repository family's oracle dead	✓	✓	
C1	Bisection assumed monotone verdicts	recovered regressions invisible	✓	✓	
C2	Declared event unit not implemented	rate off by up to $2.7\times$	✓	✓	
C3	Rollbacks produced no observation	recoveries invisible to the timeline	✓	✓	
C4	"Persists past the step boundary" undefined	one reading excludes the events of interest	✓	✓	
D1	Audit field never populated	0 tool errors reported, always	✓	✓	
D2	Malformed planner output crashed the run	33% of runs lost as task failures	✗	✓	
D3	Dependency install counted as agent work	attribution vacuous	✓	✓	
D4	Join took the first observation, not the last	failures pinned to the wrong tree	✓	✓	
D5	Executed config rebuilt from a name	manifest described a different run	✓	✓	
D6	Git handles never released	sweeps die after ~400 cells	✓	✓	
D7	Console re-walked the tree per run	presented as a dead server	✗	✓	
D8	Detection events lacked the ids attribution joined on	attributed detection always unknown	✓	✓	
D9	Absent coverage rendered as measured silence	unmeasured episodes reported as silent	✓	✓	
D10	Output-capped turns executed	half-written file; a 78-event storm	✓	✓	

290 Totals: 23 of 28 silent; 27 of 28 present under a passing suite; 6 of 28 findable only on a clean host.
291 Four further defects found after this census closed are described where they arose (Section 5.4 and
292 the repository log).

293 A.2 The defect that invalidated a conclusion

294 After nineteen fixes, every validation gate passed, the re-scoring control reproduced archived episode
295 counts exactly, and three pilots (80 runs) measured an event rate far below the pre-declared gate.
296 We drafted the conclusion the rule directed: the benchmark offered too little opportunity, so switch
297 benchmarks. The conclusion was wrong. The harness ran the agent on the host in a bare source
298 checkout, while every probe and every gate ran inside the pinned container. On the host, `import`
299 `matplotlib` in that checkout succeeds by importing the uncompiled source tree as a namespace
300 package, and everything downstream fails in ways indistinguishable from an incompetent agent.
301 Twenty-six of 28 zero-step runs traced to this. The gates had validated the measurement path and
302 never the agent's execution path. The fix was to route the agent's tools and checks through the same
303 container as the replay, and to add a per-cell environment parity check that runs before any model
304 call.

305 A.3 Mechanisms in public infrastructure

306 Class A: OpenHands issues #4235 and #7044 [31], environment construction failures against SWE-
307 bench images. Class B: UTBoost's finding that the held-out oracle is insufficient at leaderboard scale
308 [7], and, on SWE-bench Live, baseline tests that fail in the unmodified image because the calendar
309 passed a deprecation date encoded in the instance. Class C: final-state regression measurement [8].
310 Class D: the SWE-bench grader accepting forged test output on stdout [30].

311 **B Pre-declaration and analysis**

312 The event unit is one (test function, onset observation) pair with parametrised variants collapsed. The
313 gates for proceeding on a substrate were an event rate of at least 0.30 per run and a bearing fraction
314 of at least 25%. Both stacks failed the conjunction (bearing 0% and 16.2%); the gates were then
315 re-declared per (substrate, model stack) before the contrast was unblinded, and no arm reported here
316 is confirmatory. The contrast's primary endpoint is final-state contamination, paired by instance, exact
317 two-sided sign test; the co-primary is incident exposure. The analysis script was committed before
318 either comparison arm finished. The regression-gated arm was added after the contrast was unblinded
319 and is exploratory. No multiplicity adjustment was planned or applied. Budget exhaustion is scored
320 as failure. Instances whose baseline oracle records failures in the unmodified image are excluded
321 from resolve-rate comparability claims, and their dead tests are typed as missing observations for
322 event counting.